

Base teórica

Definición

El Análisis Exploratorio de Datos (EDA, por sus siglas en inglés: Exploratory Data Analysis) es una etapa fundamental en cualquier proyecto de análisis de datos o ciencia de datos. Su objetivo principal es entender la **estructura, patrones, tendencias, relaciones y anomalías** en un conjunto de datos antes de aplicar modelos predictivos o tomar decisiones.

Utilidad

- Comprender la naturaleza de los datos.
- Detectar valores atípicos (outliers).
- Identificar valores faltantes.
- Verificar supuestos estadísticos.
- Descubrir relaciones entre variables.
- Guiar la selección de técnicas estadísticas o modelos de machine learning.

Técnicas y herramientas

1. Resumen estadístico

- Media, mediana, moda
- Mínimo, máximo
- Desviación estándar, varianza
- Cuartiles

2. Visualización de datos

- Histogramas (distribución de una variable)
- Diagramas de caja (boxplots) para outliers
- Gráficos de dispersión (relaciones entre variables)
- Mapas de calor (correlación entre variables)
- Gráficos de líneas o series temporales (cuando hay datos por fecha)

3. Análisis de valores faltantes

- Detección de NaN o null
- Métodos para tratarlos (eliminación, imputación, etc.)

4. Análisis de correlación

- Coeficiente de correlación de Pearson o Spearman
- Identificar relaciones lineales o no lineales

Pasos a seguir en el EDA

(https://datos.gob.es/sites/default/files/doc/file/infografia-analisis-exploratorio-datos_0.pdf)

1. Análisis Descriptivo

- **¿Qué es?** Síntesis de la información que proporciona el conjunto de datos, extrayendo sus características más representativas.
- **¿Por qué es necesario?** Permite conocer los tipos de datos, descubrir patrones y preparar los datos para futuros análisis.
- **Tratamiento:** Aplicar funciones de estadística descriptiva para explorar la estructura del conjunto de datos, examinar los datos y las variables que presenta.

2. Re-ajuste de los tipos de variables

- **¿Qué es?** Verificar que las variables se han almacenado con el tipo de valor correspondiente.
- **¿Por qué es necesario?** Una mala codificación de las variables puede influir negativamente en la agrupación de los datos o los resultados de los análisis.
- **Tratamiento:** Aplicar la codificación apropiada para cada una de las variables.

3. Detección y tratamiento de datos ausentes

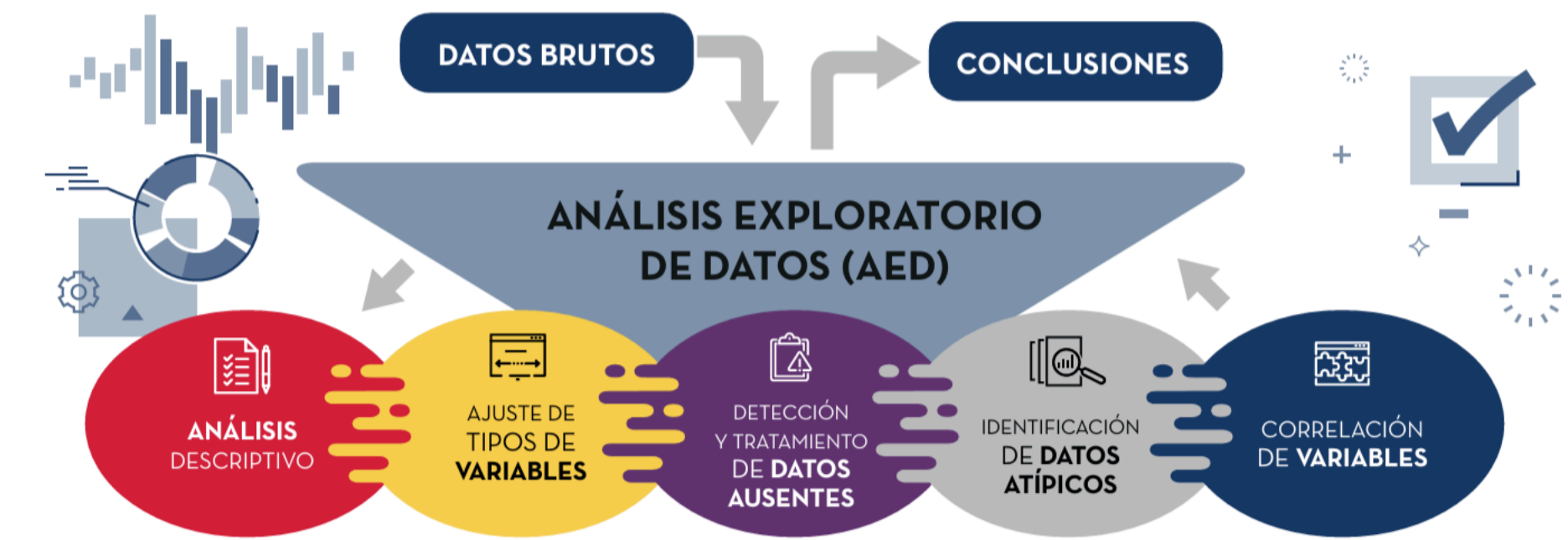
- **¿Qué es?** Identificar la falta de algunos de los datos en la variable.
- **¿Por qué es necesario?** Los datos ausentes pueden generar problemas a la hora de aplicar técnicas de machine learning, elaborar modelos predictivos, realizar análisis estadísticos o generar representaciones gráficas.
- **Tratamiento:** Existen varias maneras de tratar los valores ausentes, como por ejemplo sustituirlos por la media o la mediana, o completar los valores faltantes con el valor anterior o posterior de la columna.

4. Detección y tratamiento de datos atípicos

- **¿Qué es?** Identificar datos con valores significativamente distintos a los que presenta la variable.
- **¿Por qué es necesario su tratamiento?** Pueden modificar los resultados y restar potencia a los análisis estadísticos o técnicas de machine learning aplicadas.
- **Tratamiento:** Disminuir su influencia en análisis posteriores o, en casos muy extremos, eliminarlos del conjunto de datos.

5. Análisis de correlación de variables

- **¿Qué es?** Analizar la relación entre dos o más variables.
- **¿Por qué es necesario?** Entre otras razones, para descartar posibles variables que aporten información redundante en el conjunto de datos, ocasionando ruido en los análisis.
- **Tratamiento:** Calcular los coeficientes de correlación para las variables para detectar coeficientes cercanos a 1 ó -1.



▼ Parte experimental

Descripción del dataset

Fuente: https://colab.research.google.com/drive/1wupplEGpUB1FYxi6EUfboNKBXcEH_1m7#scrollTo=KE2OilxmTUGx&uniqifier=1

El dataset a **utilizar** es un registro de datos diarios de concentración de contaminantes registrados en las estaciones de control de la calidad del aire de la Junta de Castilla y León. (link: <https://datos.gob.es/es/catalogo/a07002862-calidad-del-aire-por-dias1>)

El dataset contiene las siguientes características (atributos):

- **Fecha:** Fecha del registro

Contaminantes atmosféricos: Cada uno de los siguientes atributos representa la concentración de un contaminante específico en el aire:

- **CO (mg/m³):** Monóxido de carbono. Se expresa en miligramos por metro cúbico.
- **NO (µg/m³):** Óxido nítrico. Se expresa en microgramos por metro cúbico.
- **NO₂ (µg/m³):** Dióxido de nitrógeno. Se expresa en microgramos por metro cúbico.
- **O₃ (µg/m³):** Ozono troposférico. Se expresa en microgramos por metro cúbico.
- **PM10 (µg/m³):** Partículas en suspensión con diámetro menor o igual a 10 micras. Expresado en microgramos por metro cúbico.
- **PM2.5 (µg/m³):** Partículas en suspensión con diámetro menor o igual a 2.5 micras. También en microgramos por metro cúbico.
- **SO₂ (µg/m³):** Dióxido de azufre. Se expresa en microgramos por metro cúbico.

Ubicación geográfica

- **Provincia:** Provincia de Castilla y León donde se encuentra la estación de medición.
- **Estación:** Nombre o identificación de la estación de control de calidad del aire.
- **Latitud:** Coordenada geográfica de latitud de la estación (en grados decimales).
- **Longitud:** Coordenada geográfica de longitud de la estación (en grados decimales).
- **Posición:** Coordenadas completas combinadas en un solo campo (latitud, longitud).

▼ 0. Carga de datos y vista general

```
1 # Cargar las librerías necesarias
2 import pandas as pd
3 import os
4
5 # Cargar los datos en un DataFrame
6 archivo = 'calidad-del-aire-datos-historicos-diarios.csv'
7 calidad_aire = pd.read_csv(archivo, sep = ';')
8
```

```
1 # Mostrar las primeras filas del DataFrame
2 calidad_aire.head(10)
```



	Fecha	CO (mg/m3)	NO (ug/m3)	NO2 (ug/m3)	O3 (ug/m3)	PM10 (ug/m3)	PM25 (ug/m3)	SO2 (ug/m3)	Provincia	Estación	Latitud	Longitud	Posición
0	2020-12-31	NaN	7.0	22.0	45.0	3.0	NaN	9.0	León	C.T.L.R. - Cuadros	42.715556	-5.638889	42.7155555556,-5.6388888889
1	2020-12-31	NaN	NaN	NaN	NaN	6.0	NaN	4.0	León	Otero	42.564444	-6.781944	42.5644444444,-6.7819444444
2	2020-12-31	NaN	6.0	16.0	64.0	4.0	NaN	2.0	Soria	Soria	41.767222	-2.480556	41.7672222222,-2.4805555556
3	2020-12-31	NaN	1.0	1.0	53.0	1.0	1.0	3.0	Palencia	C.T.G. - Compuerto	42.849167	-4.835833	42.8491666667,-4.8358333333
4	2020-12-31	NaN	2.0	21.0	41.0	NaN	NaN	NaN	Valladolid	Michelin1	41.666389	-4.715000	41.6663888889,-4.715
5	2020-12-31	NaN	3.0	14.0	52.0	9.0	NaN	2.0	Burgos	Miranda de Ebro2	42.685278	-2.940278	42.6852777778,-2.9402777778
6	2020-12-31	NaN	1.0	1.0	60.0	7.0	NaN	2.0	Burgos	Medina de Pomar	42.951667	-3.475278	42.9516666667,-3.4752777778
7	2020-12-31	NaN	5.0	3.0	57.0	4.0	NaN	3.0	Palencia	Hontoria 2 - Venta de Baños	41.948056	-4.465833	41.9480555556,-4.4658333333
8	2020-12-31	NaN	1.0	9.0	44.0	NaN	NaN	NaN	Valladolid	VALLADOLID SUR	41.611389	-4.772222	41.6113888889,-4.7722222222
9	2020-12-31	NaN	1.0	1.0	81.0	NaN	NaN	2.0	Salamanca	El Maillo	40.569444	-6.223889	40.5694444444,-6.2238888889

```
1 # Mostrar la estructura del DataFrame
2 print("Estructura del Dataframe")
3 print("="*50)
4 print(calidad_aire.info())
5
6 # Mostrar un resumen estadístico de las variables numéricas
7 print("\n Resumen estadístico de las variables numéricas")
8 print("="*50)
9 print(calidad_aire.describe())
```



Estructura del DataFrame				
=====				
<class 'pandas.core.frame.DataFrame'>				
RangeIndex: 446014 entries, 0 to 446013				
Data columns (total 13 columns):				
#	Column	Non-Null Count	Dtype	
---	-----	-----	-----	
0	Fecha	446014 non-null	object	
1	CO (mg/m3)	101158 non-null	float64	
2	NO (ug/m3)	415030 non-null	float64	
3	NO2 (ug/m3)	413497 non-null	float64	
4	O3 (ug/m3)	275414 non-null	float64	
5	PM10 (ug/m3)	344579 non-null	float64	
6	PM25 (ug/m3)	53784 non-null	float64	
7	SO2 (ug/m3)	356277 non-null	float64	
8	Provincia	446014 non-null	object	
9	Estación	446014 non-null	object	
10	Latitud	445788 non-null	float64	
11	Longitud	445788 non-null	float64	
12	Posición	445788 non-null	object	
dtypes: float64(9), object(4)				
memory usage: 44.2+ MB				
None				
Resumen estadístico de las variables numéricas				
=====				
	CO (mg/m3)	NO (ug/m3)	NO2 (ug/m3)	O3 (ug/m3) \

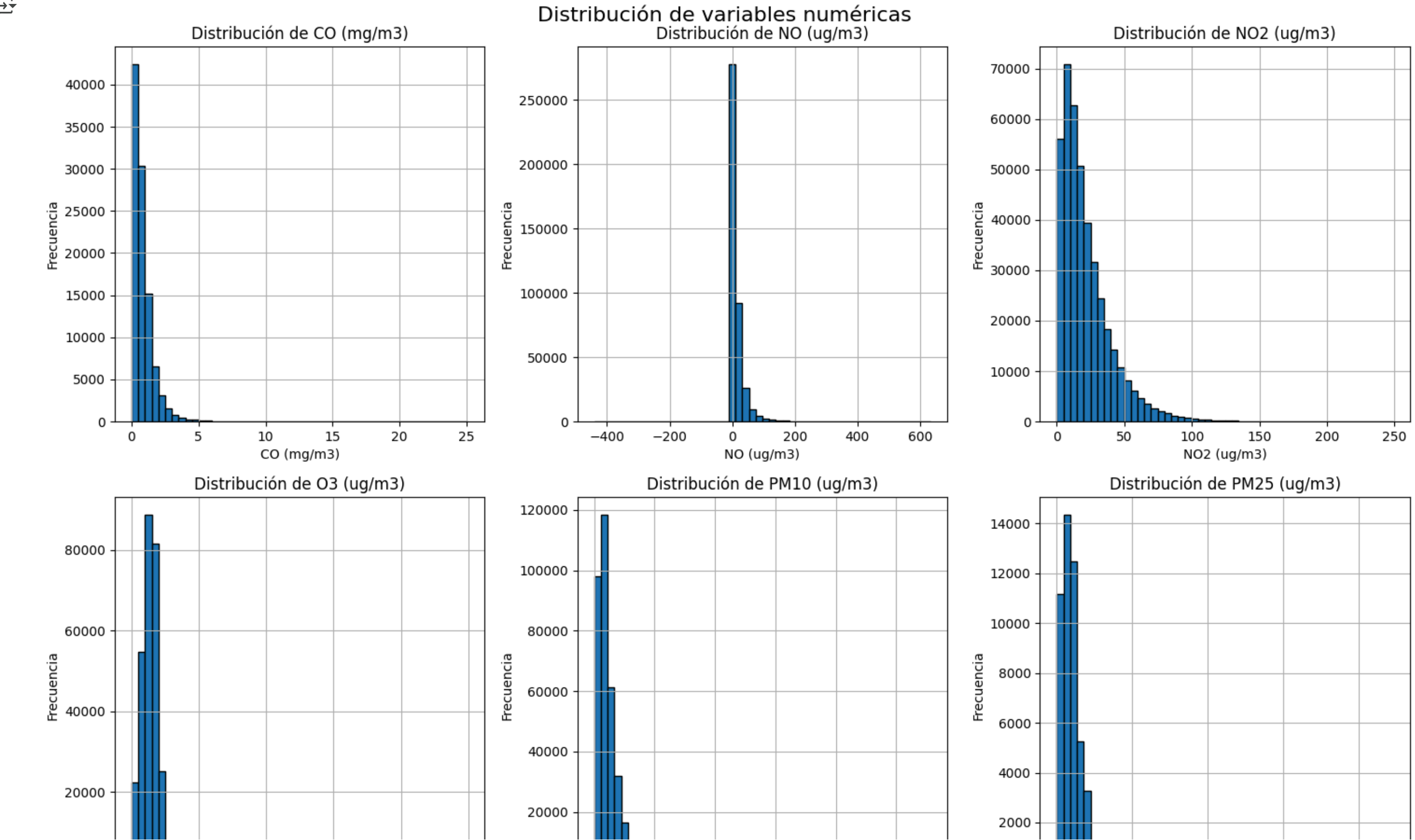
count	101158.000000	415030.000000	413497.000000	275414.000000
mean	0.854624	13.225808	21.409154	52.619754
std	0.785226	21.970729	19.108434	23.221958
min	0.000000	-441.000000	0.000000	0.000000
25%	0.300000	2.000000	8.000000	37.000000
50%	0.700000	5.000000	16.000000	54.000000
75%	1.100000	15.000000	29.000000	68.000000
max	25.100000	634.000000	249.000000	999.000000

	PM10 (ug/m3)	PM25 (ug/m3)	SO2 (ug/m3)	Latitud \
count	344579.000000	53784.000000	356277.000000	445788.000000
mean	22.694662	13.677172	9.092801	42.151547
std	17.919319	15.895495	13.790750	0.665501
min	0.000000	0.000000	-791.000000	38.938333
25%	11.000000	5.000000	2.000000	41.645556
50%	18.000000	9.000000	5.000000	42.542778
75%	29.000000	15.000000	11.000000	42.688056
max	557.000000	223.000000	364.000000	43.603333

	Longitud
count	445788.000000
mean	-5.178965
std	1.121920
min	-6.781944
25%	-6.483889
50%	-4.909167
75%	-4.538333
max	-2.466667

1. Distribución de las variables numéricas

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Seleccionar solo las columnas numéricas
5 columnas_numericas = calidad_aire.select_dtypes(include=[np.number]).columns
6
7 # Calcular el número de filas y columnas para el subplot
8 n = len(columnas_numericas)
9 nrows = 3
10 ncols = min(n, 3)
11
12 # Crear la figura y los subplots
13 fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(15, 5*nrows))
14 fig.suptitle('Distribución de variables numéricas', fontsize=16)
15
16 # Aplanar el array de ejes en caso de que sea 2D
17 axes = axes.flatten() if n > 3 else [axes]
18
19 # Crear histogramas para cada variable numérica
20 for i, col in enumerate(columnas_numericas):
21     ax = axes[i]
22     calidad_aire[col].hist(ax=ax, bins=50, edgecolor='black')
23     ax.set_title(f'Distribución de {col}')
24     ax.set_xlabel(col)
25     ax.set_ylabel('Frecuencia')
26
27 # Ocultar subplots vacíos si los hay
28 for j in range(i+1, len(axes)):
29     fig.delaxes(axes[j])
30
31 plt.tight_layout()
32 plt.show()
33
```



2. Ajuste de las variables categóricas

```
1 # Ajustar el tipo de la variable Fecha
2 calidad_aire['Fecha'] = pd.to_datetime(calidad_aire['Fecha'], errors='coerce')
```

```
3
4 # Ajustar el tipo de la variable Provincia
5 print(calidad_aire['Provincia'].unique())
6 calidad_aire['Provincia'] = calidad_aire['Provincia'].astype('category')
7
8 # Ajustar el tipo de la variable Estación
9 print(calidad_aire['Estación'].unique())
10 calidad_aire['Estación'] = calidad_aire['Estación'].astype('category')
```

 ['León' 'Soria' 'Palencia' 'Valladolid' 'Burgos' 'Salamanca' 'Zamora' 'Avila' 'Segovia' 'Madrid']

['C.T.L.R. - Cuadros' 'Otero' 'Soria' 'C.T.G. - Compuerto' 'Michelin1' 'Miranda de Ebro2' 'Medina de Pomar' 'Hontoria 2 - Venta de Baños' 'VALLADOLID SUR' 'El Maillo' 'Arco de ladrillo II' 'Puente Poniente-Mº Luisa Sánchez' 'Aranda de Duero 2' 'Lario' 'Palencia 3' 'Zamora 2' 'Avila II' 'Salamanca6' 'Renault2' 'Tudela Veguin-Tudela Veguin' 'Hontoria 1 - Poblado' 'Salamanca5' 'Vega Sicilia' 'Burgos4' 'Segovia 2' 'La Robla' 'Valderas' 'Michelin2' 'Muriel de la Fuente' 'Renault3' 'Leon 4' 'Ponferrada4' 'Renault1' 'Torale de los Vados' 'C.T.G. - Villalba' 'Burgos1' 'C.T.L.R. - Naredo' 'Carracedelo' 'Guardo' 'Renault4' 'La rubia II' 'Medina del Campo' 'C.T.L.R. - Ventosilla' 'Leon1' 'C.T.Compostilla-Cortiguera' 'C.T.Compostilla-Congosto' 'C.T.Compostilla-Compostilla' 'C.T.Compostilla-Santa Marina' 'C.T.Compostilla-Villaverde' 'NH3' 'Miranda de Ebro1' 'Puente Regueral' 'San Martín de Valdeiglesias' 'C.T.Anllares - Lillo' 'C.T.Anllares - SusaÑe' 'C.T.Anllares - Palacios del Sil' 'C.T.Anllares - Anllares' 'C.T.Anllares - Hospital del Sil' 'C.T.Compostilla-Sancedo' 'C.T.Compostilla-Cueto' 'C.T.Compostilla-San Miguel' 'Salamanca4' 'Burgos5' 'Leon3' 'Labradores II' 'Cementerio' 'Velilla del Río Carrion' 'Ponferrada5' 'Ponferrada1' 'Salamanca2' 'Venta de Baños' 'Santa Teresa' 'Leon2' 'Burgos3' 'C.T.Anllares - Páramo del Sil' 'C.T.Anllares - Sorbeda' 'C.T.Anllares - Anllarinos' 'C.T.L.R. - La Robla' 'Zamora' 'Avila' 'Segovia' 'Aranda de Duero' 'Palencia2' 'Salamanca3' 'Burgos2' 'Ponferrada2' 'Ponferrada3' 'Miranda de Ebro3' 'Palencia1' 'Salamanca1' 'Arco Ladrillo I']

3. Análisis de datos ausentes (faltantes)

```
1 # Devuelve un DataFrame booleano
2 calidad_aire.isna()
3
4 # Devuelve True si hay al menos un valor ausente
5 calidad_aire.isna().any().any()
6
7 # Devuelve el número total de NaN que presenta el DataFrame
8 print(calidad_aire.isna().sum().sum())
9 # Devuelve el % de valores perdidos
10 print(calidad_aire.isna().mean().mean())
11
12 # Detección del número de valores perdidos en cada una de las columnas
13 calidad_aire.isna().sum()
14
15 # Detección del % de valores perdidos en cada una de las columnas
16 calidad_aire.isna().mean().round(2)
```

 1163037
0.20058649418041727

0	
Fecha	0.00
CO (mg/m3)	0.77
NO (ug/m3)	0.07
NO2 (ug/m3)	0.07
O3 (ug/m3)	0.38
PM10 (ug/m3)	0.23
PM25 (ug/m3)	0.88
SO2 (ug/m3)	0.20
Provincia	0.00
Estación	0.00
Latitud	0.00
Longitud	0.00
Posición	0.00

dtype: float64

Tratamiento de datos ausentes (eliminación e imputación por la media)

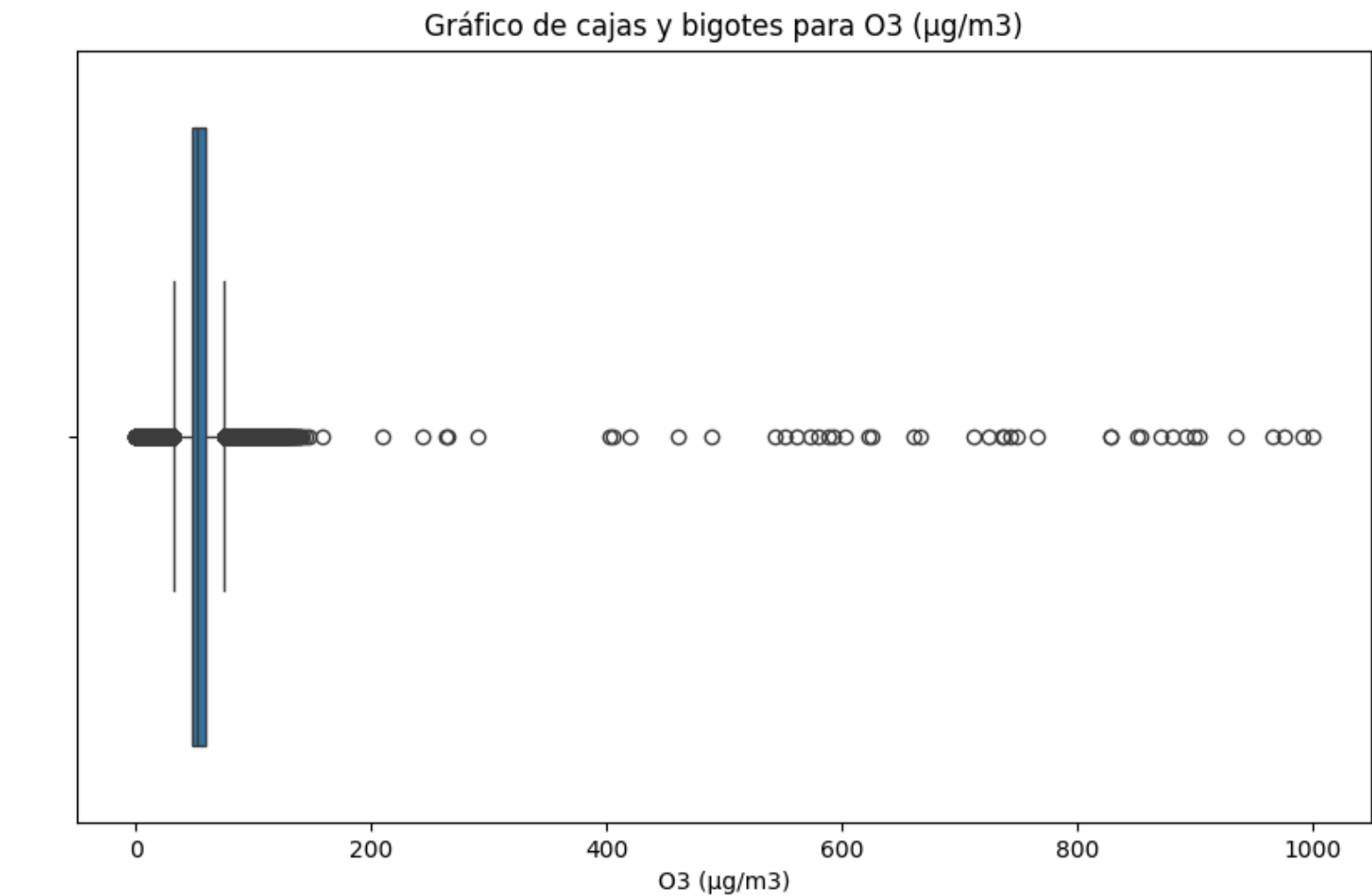
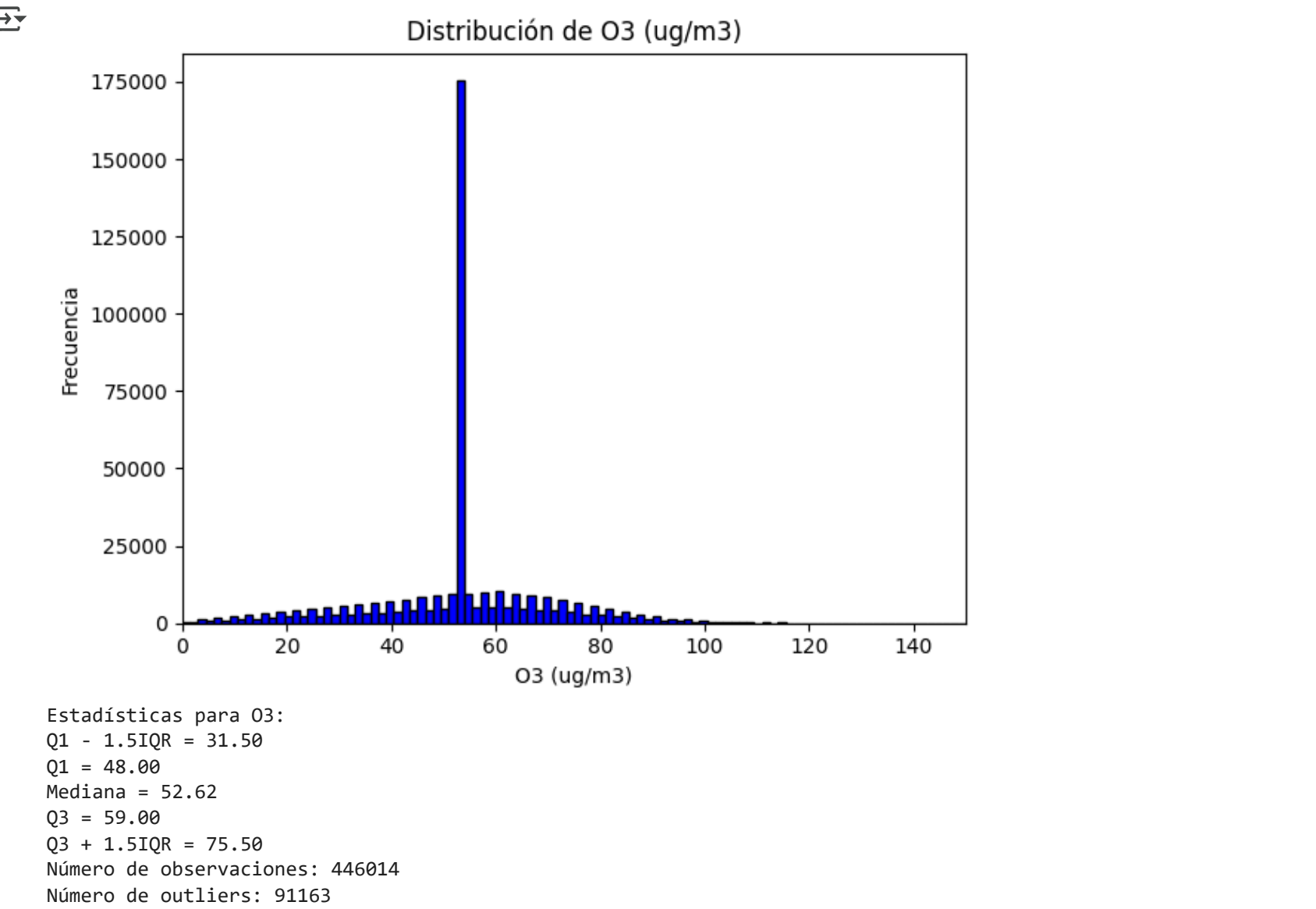
```
1 # Guardar dataset original
2 calidad_aire_original = calidad_aire.copy()
3
4 # Eliminación de las variables que presentan un % de NaN superior al 50%
5 calidad_aire = calidad_aire.loc[:, calidad_aire.isna().mean() < 0.5]
6 print(f" Tras esta operación, contamos con {len(calidad_aire.columns)} columnas")
7
8 # Seleccionamos las variables numéricas
9 columnas_numericas = calidad_aire.select_dtypes(include=[np.number]).columns
10
11 # Calculamos la media para cada una de las variables numéricas sin tener en cuenta los NaN
12 cols_mean = calidad_aire[columnas_numericas].mean()
13
14 # Sustituimos los valores NaN por la media correspondiente a cada variable
15 calidad_aire[columnas_numericas] = calidad_aire[columnas_numericas].fillna(cols_mean)
```

 Tras esta operación, contamos con 11 columnas

4. Detección de datos atípicos en variablrs numéricas (Histograma, gráfico de cajas y gráfico de bigotes)

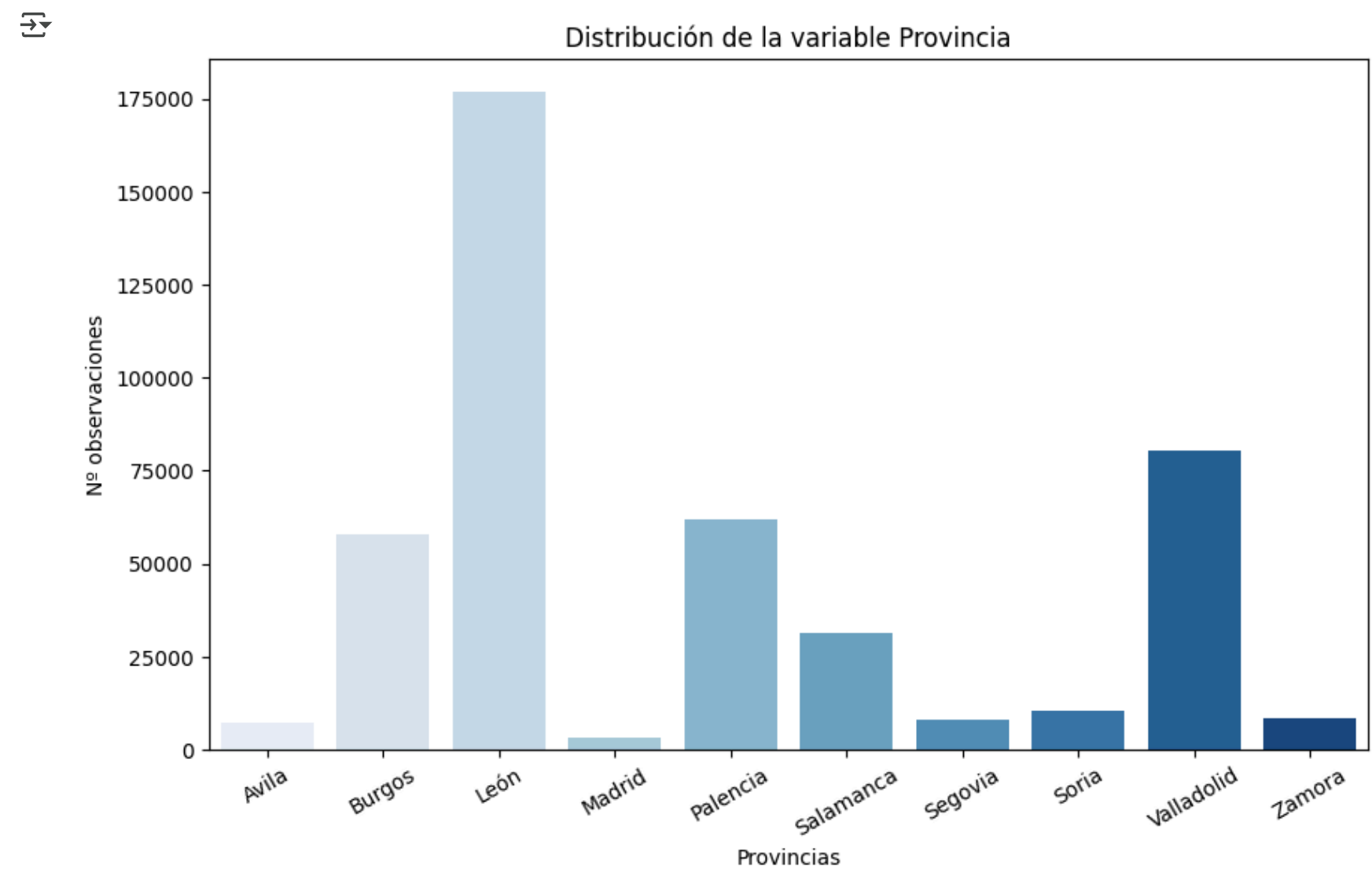
```
1 # Histograma para analizar la distribución de ozono (O3)
2 plt.hist(calidad_aire['O3 (ug/m3)'], bins=100, range=(0, 150), color='blue', edgecolor='black')
3 plt.title('Distribución de O3 (ug/m3)')
4 plt.xlabel('O3 (ug/m3)')
5 plt.ylabel('Frecuencia')
6 plt.xlim(0,150)
7 plt.tight_layout()
8 plt.show()
```

```
9
10 # Estadísticas necesarias para reproducir el gráfico de cajas y bigotes
11 import seaborn as sns
12 Q1 = calidad_aire['O3 (ug/m3)'].quantile(0.25)
13 Q3 = calidad_aire['O3 (ug/m3)'].quantile(0.75)
14 IQR = Q3 - Q1
15 lower_bound = Q1 - 1.5 * IQR
16 upper_bound = Q3 + 1.5 * IQR
17
18 print(f"Estadísticas para O3:")
19 print(f"Q1 - 1.5IQR = {lower_bound:.2f}")
20 print(f"Q1 = {Q1:.2f}")
21 print(f"Mediana = {calidad_aire['O3 (ug/m3)'].median():.2f}")
22 print(f"Q3 = {Q3:.2f}")
23 print(f"Q3 + 1.5IQR = {upper_bound:.2f}")
24 print(f"Número de observaciones: {len(calidad_aire['O3 (ug/m3)'])}")
25 print(f"Número de outliers: {sum((calidad_aire['O3 (ug/m3)'] < lower_bound) | (calidad_aire['O3 (ug/m3)'] > upper_bound))}")
26
27 # Construcción del gráfico de cajas y bigotes
28 plt.figure(figsize=(10, 6))
29 sns.boxplot(x=calidad_aire['O3 (ug/m3)'])
30 plt.title('Gráfico de cajas y bigotes para O3 (µg/m3)')
31 plt.xlabel('O3 (µg/m3)')
32 plt.show()
33
```



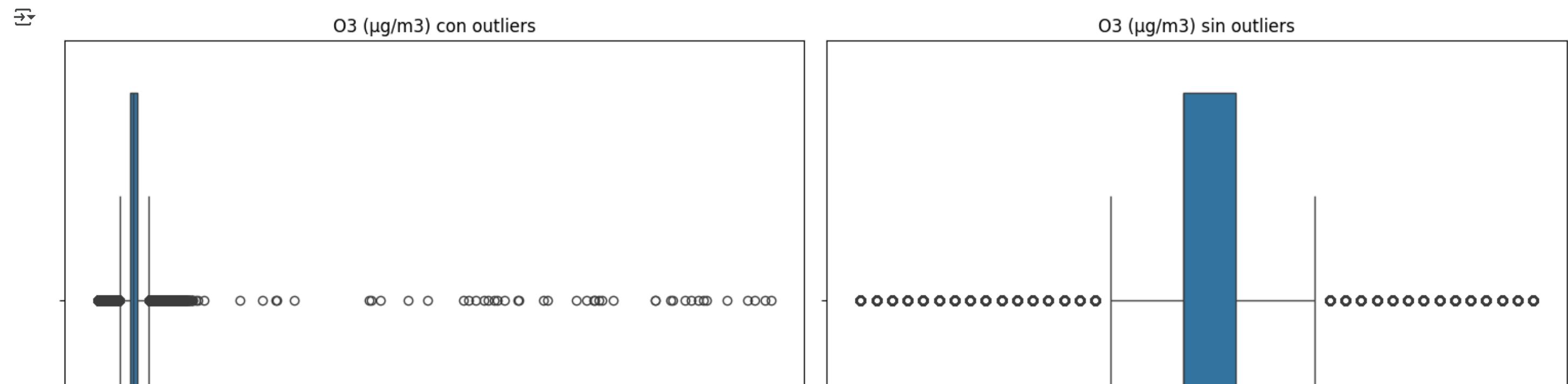
▼ Detección de valores atípicos en variables categóricas

```
1 # Número de categorías que presenta la variable Provincia
2 categoria_counts = calidad_aire['Provincia'].value_counts()
3
4 # Construcción del gráfico de barras para la variable Provincia
5 plt.figure(figsize=(10, 6))
6 sns.barplot(x=categoria_counts.index, y=categoria_counts.values, palette='Blues', hue=categoria_counts.index, legend=False)
7 plt.xlabel('Provincias')
8 plt.ylabel('Nº observaciones')
9 plt.xticks(rotation=30)
10 plt.title('Distribución de la variable Provincia')
11 plt.show()
12
```



▼ Tratamiento de valores atípicos en variables numéricas

```
1 # Se genera una nueva tabla que no contiene los valores identificados como atípicos
2 calidad_aire_NoOut = calidad_aire[(calidad_aire['O3 (ug/m3)'] >= lower_bound) &
3 (calidad_aire['O3 (ug/m3)'] <= upper_bound)]
4
5 # Construcción de los gráficos de cajas y bigotes
6 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
7
8 sns.boxplot(x=calidad_aire['O3 (ug/m3)'], ax=ax1)
9 ax1.set_title('O3 (µg/m3) con outliers')
10 ax1.set_xlabel('O3 (µg/m3)')
11
12 sns.boxplot(x=calidad_aire_NoOut['O3 (ug/m3)'], ax=ax2)
13 ax2.set_title('O3 (µg/m3) sin outliers')
14 ax2.set_xlabel('O3 (µg/m3)')
15
16 plt.tight_layout()
17 plt.show()
18
```



▼ Tratamiento de valores atípicos en variables categóricas

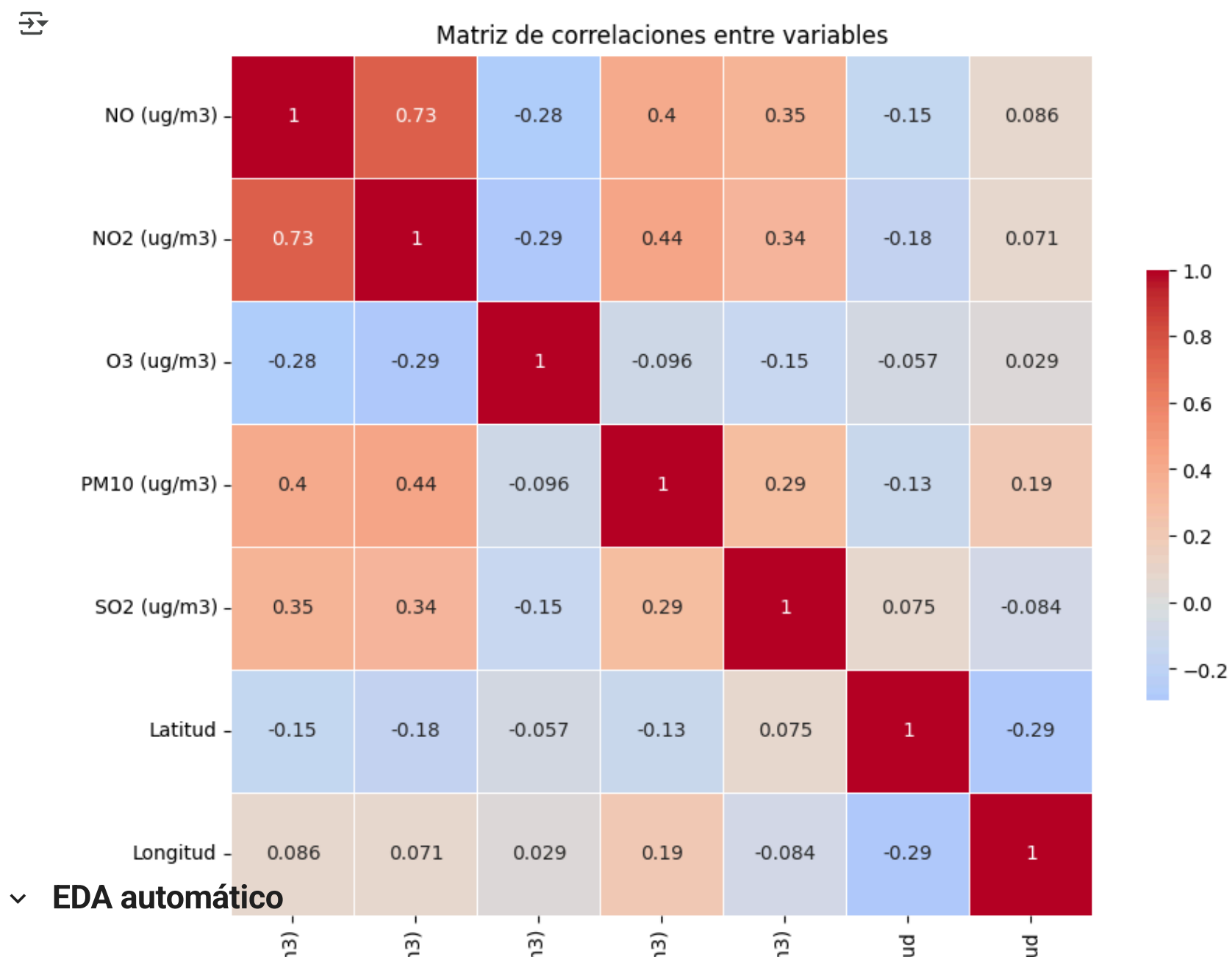
```
1 # Creamos una copia explícita del DataFrame excluyendo las filas de Madrid
2 calidad_aire_SM = calidad_aire[calidad_aire['Provincia'] != 'Madrid'].copy()
3
4 # Eliminamos las categorías no utilizadas
5 calidad_aire_SM['Provincia'] = calidad_aire_SM['Provincia'].astype('category').cat.remove_unused_categories()
6
7 # Verificamos la eliminación de la categoría "Madrid"
8 print(calidad_aire_SM['Provincia'].cat.categories)
```

🔗

```
Index(['Avila', 'Burgos', 'León', 'Palencia', 'Salamanca', 'Segovia', 'Soria',
      'Valladolid', 'Zamora'],
      dtype='object')
```

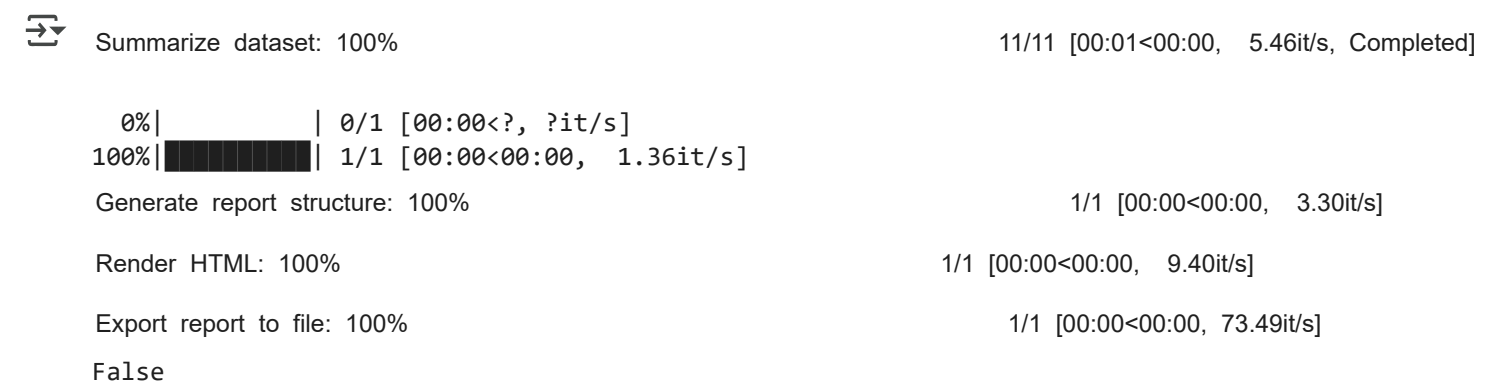
▼ 5. Análisis de correlación de variables

```
1 num_variables = calidad_aire.select_dtypes(include=[np.number])
2
3 # Calculamos la matriz de coeficientes de correlación entre las variables numéricas
4 correlacion = num_variables.corr()
5
6 # Configuración del gráfico de correlación
7 plt.figure(figsize=(10, 8))
8
9 # Gráfico de correlaciones utilizando un mapa de calor
10 sns.heatmap(correlacion, annot=True, cmap='coolwarm', center=0, square=True, linewidths=.5, cbar_kws={"shrink": .5})
11
12 plt.title('Matriz de correlaciones entre variables')
13 plt.show()
14
```



EDA automático

```
1 # Instalación de paquetes y dependencias
2 !pip install -q setuptools # gestionar dependencias en Python
3 !pip install -q --upgrade ydata-profiling #librería que genera el reporte automático EDA
4 !pip install -q ipywidgets # utilidad para mostrar elementos interactivos
5
6 # Importación de librerías
7 import pandas as pd
8 import webbrowser
9 from ydata_profiling import ProfileReport
10 import os
11
12 # Carga del archivo CSV
13 calidad_aire_original = pd.read_csv('calidad-del-aire-datos-historicos-diarios.csv')
14
15 # Generación del reporte automático
16 report = ProfileReport(calidad_aire_original, title='EDA automático', explorative=True)
17
18 # Guardar el reporte como archivo HTML
19 report_file = os.path.abspath('reporte_calidad_aire.html')
20 report.to_file(report_file)
21
22 # Abrir el archivo HTML en el navegador web
23 # (si el navegador no se abre automáticamente, descargar el archivo html y cargarlo desde el computador)
24 webbrowser.open(f'file://{report_file}')
```



Ejercicios

1. Realice un EDA con el dataset de Data set de Ordenes de Compra y ordenes de Servicio años 2025 [MPSI] de la plataforma Nacional de Datos Abiertos del Perú (link: <https://datosabiertos.gob.pe/dataset/%C3%B3rdenes-de-compra-y-ordenes-de-servicio-a%C3%B1os-2025-municipalidad-provincial-san-ignacio-0#>)

1 Empieza a programar o a crear código con IA.

2. Investigue herramientas de EDA-automático como Sweetviz, AutoViz, D-Tale, DataPrep.EDA, Lux, etc. Haga pruebas con dos de ellas, analice los datos obtenidos y realice una comparativa de ambas herramientas.

1 Empieza a programar o a crear código con IA.